Language Models and Author Identification

LING 5832, Assignment 2 · Yiyuan Jia · September 21, 2026

Code: https://github.com/yiyuan1840/ling-5832/tree/main/assignments/A2

1. What I built

A BPE tokenizer using the Hugging Face `tokenizers` library, bigram and trigram models with add k smoothing, perplexity, and a classifier that guesses which of the two authors wrote a passage.

The classifier got **255 out of 255** right on 50 word passages. That seemed too good, so I tested whether it was cheating, and it still got **97.6%** after I removed the two easiest clues. More on that in section 6.

Three things surprised me while doing this:

- add 1 smoothing, the textbook default, is almost three times worse than a smaller k
- the trigram model is worse than the bigram, which I did not expect at all
- perplexity looks like it says a small vocabulary is best, and that turns out to be a trap

2. The data

Table 1. The two books, after I stripped the Project Gutenberg wrapper off the Doyle file.

author	sentences	words	held out	word types
Tolkien, *The Hobbit*	4,929	95,940	493	6,128
Conan Doyle, *The Lost World*	3,705	75,759	371	7,760
word types in both books: 3,354				

I did three things to the text before using it.

First, the Doyle file still had about 379 lines of Project Gutenberg licence attached. I removed it. If I had left it in, words like *Gutenberg* and *eBook* would end up in the vocabulary, and they only show up in one of the two books. The classifier could then tell the books apart by their file wrapper instead of the writing, which is not what I want to measure.

Second, I split the text into sentences by cutting after a full stop, question mark or exclamation mark followed by a space. This is rough. It breaks on things like *Mr. Baggins*.

Third, I held back the last 10% of each book to test on.

That last one is the weakest part of my setup and I want to be upfront about it. The end of The Hobbit is the Battle of Five Armies, which has a different cast and a different feel from the opening chapters. So my perplexity numbers are partly measuring how well the model handles the ending, not just how hard the book is. Taking chunks from all through the book would be better.

3. Tokenization

I used `models.BPE` and `trainers.BpeTrainer` and trained the vocabulary on the two books. No pretrained tokenizer anywhere. I wrapped all of it in three functions of my own, `train_tokenizer`, `encode` and `pad`, so the rest of my code only ever sees lists of numbers and never touches a Hugging Face object directly. I train one vocabulary on both authors' training text, and never on the text I test with.

I compared three ways of deciding where BPE is allowed to cut.

Table 2. Vocabulary 4,000. Perplexity is on held out Tolkien.

strategy	tokens per word	types with punctuation stuck on	perplexity
whitespace only	1.29	533	324.4
whitespace and punctuation	1.35	**0**	234.3
byte level	1.44	9	**198.0**

The third column is what made my decision. If you only split on whitespace, BPE is free to glue a quote mark onto a word, and you end up with 533 vocabulary entries like `morning!"`. That matters here because the two books use different quote characters. Tolkien's edition uses curly ones and Doyle's uses straight ones. So an entry like that is really recording which book the text came from, not who wrote it. Splitting off every punctuation mark first makes that impossible and the count drops to zero.

Byte level actually got the best perplexity and I still did not pick it, so I should explain that. It puts nine glued types back, it makes sequences 7% longer, and its advantage is on perplexity, which I stopped trusting for the reason in section 5. If perplexity were the only thing I cared about, byte level would win.

One thing I got wrong at first. Lecture 3 says BPE has no out of vocabulary words because the vocabulary starts from UTF 8. I assumed that meant it just works. It does not. I tested it on `café`, `你好`, `🐞` and `naïve`, and the byte level tokenizer gave 11 unknown tokens and could not reproduce the input. The reason is that the starting alphabet only contains characters that were in the training corpus, and two English novels do not contain `é`. Adding `initial_alphabet=ByteLevel.alphabet()` puts all 256 byte characters in and fixes it completely. So the lecture is right about the algorithm but not about what the library does unless you ask.

4. The n gram models

Training is just counting. One pass over the text, no repeats, nothing to adjust. This surprised me, because I expected training to look like a neural network with weights being nudged. There are no weights here at all. The counts are the model.

I keep two things, both keyed on vocabulary index. `counts` maps a context to a tally of what came after it, and `totals` maps a context to how many times it showed up. A context is a tuple of n − 1 numbers so the same code works for bigrams and trigrams. I could not use a plain table for this. At a vocabulary of 4,000 a bigram table would need 16 million cells and a trigram table 64 billion.

For smoothing I used add k, so P(w | c) = (count(c,w) + k) / (total(c) + kV). The assignment does not say what some of these should be, so here is what I picked and why.

V is the vocabulary size minus one. Every number can be predicted except the sentence start marker, which only ever appears at the front.

The bottom of the fraction comes from the n gram counts, not from how many times the context word appears in the book. Those are slightly different, because a word at the end of a sentence appears without ever being a context. I nearly used the wrong one.

The end marker counts as a prediction. Without it the model cannot say a sentence is over, and the probabilities do not add up to 1.

I checked all three by adding up the probability of every possible next word for a given context. It comes to exactly 1.0000000000 every time, for contexts the model has seen and ones it has not. An untrained model gives perplexity 3,999, which is V, which is what it should be if it truly knows nothing.

For padding I put n − 1 start markers at the front and one end marker at the back. This means a bigram and a trigram make exactly the same number of predictions on the same text, 162,151 for my test set. That matters because otherwise I could not put their perplexities in the same table and compare them.

I used natural logs. The textbook says at equation 3.42 that the base is up to you, and I checked that perplexity comes out the same either way.

5. Perplexity

Table 3. Perplexity on held out Tolkien. Accuracy is on 50 word passages.

n	k	perplexity	contexts never seen	accuracy
2	1.0	678.1	0.2%	100.0%
2	0.1	306.2	0.2%	100.0%
2	**0.01**	**234.3**	0.2%	100.0%
2	0.001	310.7	0.2%	99.6%
3	1.0	2211.0	29.7%	98.8%
3	0.1	1313.5	29.7%	98.4%
3	0.01	891.8	29.7%	98.8%
3	0.001	899.4	29.7%	98.0%

Add 1 is a bad default. At k = 1 the bigram gets 678. At k = 0.01 it gets 234. You can see the problem in one example. The real probability of *the dwarves* is 0.0302, and add 1 drops it to 0.0178. That is throwing away 41% of the probability of something I saw 171 times, just to leave room for thousands of pairs I never saw once. Going below 0.01 makes things worse again, so there is a real sweet spot rather than smaller always being better.

The trigram is worse than the bigram at every value of k. This bothered me, because a trigram looks back further so it should know more. The fourth column is the answer. A bigram runs into a context it has never seen 0.2% of the time. A trigram does it 29.7% of the time, because it needs to have seen that exact *pair* of words before. When that happens the formula falls back to treating every word as equally likely, which is a terrible guess, and it is making it on almost a third of all predictions. Apparently the normal fix is backing off to a shorter context, which is beyond what this assignment asks for.

Table 4. Vocabulary size. See below before reading the perplexity column.

requested	realized	tokens per word	perplexity	accuracy
500	500	2.00	61.6	99.2%
1,000	1,000	1.71	89.4	99.6%
2,000	2,000	1.50	140.0	99.6%
4,000	4,000	1.35	234.3	**100.0%**
8,000	8,000	1.26	362.2	99.6%
16,000	15,700	1.20	528.4	99.2%

This table almost fooled me. It looks like a vocabulary of 500 is eight times better than 16,000. It is not. The perplexity here is per token, and a small vocabulary chops each word into more tokens, 2.00 against 1.20. Each of those pieces is easier to guess than a whole word. So the model is answering more questions that are each easier, which makes the number look good without the model being any better. You can only compare per token perplexity when the vocabulary is the same size, which is why Table 3 keeps it fixed. Accuracy does not have this problem and it stays between 99.2% and 100%, with the best at 4,000. So I picked 4,000 based on accuracy, not perplexity. If I did this again I would also compute perplexity per word, which would make the column mean something.

Small thing worth noting: I asked for 16,000 and got 15,700. The books run out of useful merges to make. So code should always ask the tokenizer how big the vocabulary actually is, especially since that number goes into the smoothing formula.

6. Author identification

I train two models on one shared vocabulary, one per author, score a passage with both, and pick whichever is less surprised.

The shared vocabulary matters. If each author had their own, the same passage would turn into two different lists of numbers of two different lengths, and comparing the two scores would not mean anything. On top of that, whichever tokenizer had seen that author's text would chop it into fewer pieces and get an unfair advantage that has nothing to do with writing style.

I divide each score by the number of predictions. This does not change who wins, since both models see the same passage cut the same way, but it makes the gap between the two scores comparable between a short passage and a long one.

Table 5. Perplexity of each model on each author's held out text.

model	on Tolkien	on Doyle
Tolkien	**234.3**	753.6
Doyle	751.5	**275.9**

Each model is about three times more surprised by the other author than by its own, and that gap is basically the whole thing. The reason this is a valid way to classify comes from Bayes' rule. What I want is P(author | text), but the models give me P(text | author). When you compare two authors, the P(text) term is the same on both sides and cancels, and if you assume both authors are equally likely to start with, that cancels too. What is left is just the two model scores.

Table 6. Accuracy at different passage lengths, and with the easy clues taken away.

what the model got to see	10 words	25 words	50 words	100 words
everything	94.4%	98.1%	100.0%	100.0%
quotes and dashes made the same	91.9%	96.9%	99.6%	100.0%
character names replaced	93.5%	97.4%	99.2%	100.0%
both	90.1%	95.7%	97.6%	100.0%

Getting 100% made me suspicious rather than pleased, so I checked what the model was actually using. Two things in these books have nothing to do with how the authors write. The editions punctuate differently, and the two casts share no names, so a model that spots *Bilbo* against *Challenger* is really just a name detector.

Taking both away costs 4.3 points at 10 words and nothing at all at 100 words. So the clues help most when there is the least other evidence, which makes sense. There is genuine signal in the writing itself, but I do not think reporting 100% on its own would be a fair thing to say.

Two other things I noticed. Changing k barely moves the accuracy even though it changes perplexity a lot, because k shifts both models the same way and only the order matters. And passage length matters far more than any setting I tried.

I also looked at the passages where the two scores were closest. The smallest gaps at 50 words were 0.043 and 0.047 nats, and one of those was a Tolkien *song*, which makes sense because verse does not sound like the prose the model was trained on. All the close

ones were still right.

7. Problems I ran into and things I noticed

1. **The bottom of the add k fraction is easy to get wrong.** If you use how often the word appears instead of how often it was a context, everything still runs and the numbers still look reasonable. Adding up the probabilities and checking they come to exactly 1 is what catches it. I would write that check first next time.
2. **A number can look comparable when it is not.** Table 4 reads like a clear recommendation and it is the opposite. Anything that changes how many predictions are being made needs a second look.
3. **The library does not always do what the lecture describes.** The out of vocabulary thing needed an argument I would never have thought to pass if I had not actually tested the claim.
4. **Getting everything right was a reason to dig, not to stop.** Table 6 is the real result of this assignment, not the 100%.
5. **Things I know are still wrong.** My test set is the tail of each book instead of a sample from all through it. My sentence splitter breaks on abbreviations. And the scanned Tolkien text has OCR damage in it, like `1/1 ine` where it should say *wine*, which the Doyle text does not have, so that is one more small clue the classifier could be using that I did not remove.
6. **I do not think this would generalize.** Both models only ever saw one book. A lot of what they know is the subject matter and the characters. If I ran this on passages from other books by the same two authors I would expect it to do a lot worse than Table 6 suggests.

8. Code, and how much of it is mine

The assignment asks me to describe any pre existing or AI generated code. I worked on this with Claude (Anthropic) the whole way through, so a vague acknowledgement would not be honest. Here is the actual split.

Mine. All the functions that do the real work: `train_tokenizer` and `pad` in `tokenizer.py`, `fit`, `prob`, `neg_log_prob`, `cross_entropy` and `perplexity` in `ngram.py`, `score` in `authorid.py`, and `split_sentences` in `corpus.py`. I typed every one of these out by hand in a scratch file first, following a tutorial that explained the idea but did not give me the answer, and only moved them into the real modules once they worked.

Claude's. The scaffolding around my code: the empty module skeletons and their docstrings, the file loading and data splitting in `corpus.py`, the code in `main.py` that wires the four parts together, the `experiments.py` script that generated every table above, and the tutorial notes in `notes/`. Claude also ran the experiments and wrote the first draft of this report from the results.

How it went. Claude acted as a tutor instead of just writing it for me. It explained each idea from scratch, told me what each function needed to do and why, and then checked what I wrote. It caught real mistakes in my code, including a function that silently returned `None` when given input it did not handle, and a place where I wrote `self.model` when I meant `self`. I caught mistakes in its explanations too. It told me the order of two pre tokenizers mattered, and when I tested it, it did not. It also had me write a loop I had already written somewhere else, which we deleted. Every number in this report was regenerated from the final version of the code rather than copied from an earlier run.

No pretrained tokenizer and no existing n gram library is used anywhere. The only outside package is `tokenizers` *0.23.2. Python 3.13.6. All of the code is at https://github.com/yiyuan1840/ling-5832/tree/main/assignments/A2 and every number above can be reproduced with* `uv run main.py` *and* `uv run src/experiments.py`*.*